

第十课预习报告

ptr_ops.c

- 作用: 演示各种指针运算（加法、减法、自增自减）及其与数组间的关系。
- 功能:
 1. 定义整型数组 `urn[5]`，并通过多个指针变量操作数组；
 2. 展示取地址、解引用、指针加减整数的效果；
 3. 对两个指针做差，说明结果表示数组元素间的距离；
 4. 理解自增、自减对指针指向内存位置的影响；
 5. 注意其中输出格式字符串有手误：`ptr4` 应为 `ptr1`。
- 语义错误:
 - 在 `printf("ptr1 + 4 = %p, *(ptr4 + 3) = %d\n", ...)` 语句中，`ptr4` 未定义，应改为 `ptr1`。
- 演示效果:

```
1 // ptr_ops.c -- pointer operations
2 #include <stdio.h>
3 int main(void)
4 {
5     int urn[5] = {100,200,300,400,500};
6     int * ptr1, * ptr2, *ptr3;
7
8     ptr1 = urn;           // assign an address to a pointer
9     ptr2 = &urn[2];       // ditto
10    // dereference a pointer and take
11    // the address of a pointer
12    printf("pointer value, dereferenced pointer, pointer address:\n");
13    printf("ptr1 = %p, *ptr1 = %d, &ptr1 = %p\n",
14           ptr1, *ptr1, &ptr1);
15
16    // pointer addition
17    ptr3 = ptr1 + 4;
18    printf("\nadding an int to a pointer:\n");
19    printf("ptr1 + 4 = %p, *(ptr1 + 3) = %d\n", // 修正 ptr4 为 ptr1
20           ptr1 + 4, *(ptr1 + 3));
```

```

21     ptr1++;          // increment a pointer
22     printf("\nvalues after ptr1++:\n");
23     printf("ptr1 = %p, *ptr1 = %d, &ptr1 = %p\n",
24           ptr1, *ptr1, &ptr1);
25     ptr2--;          // decrement a pointer
26     printf("\nvalues after --ptr2:\n");
27     printf("ptr2 = %p, *ptr2 = %d, &ptr2 = %p\n",
28           ptr2, *ptr2, &ptr2);
29     --ptr1;          // restore to original value
30     ++ptr2;          // restore to original value
31     printf("\nPointers reset to original values:\n");
32     printf("ptr1 = %p, ptr2 = %p\n", ptr1, ptr2);
33     // subtract one pointer from another
34     printf("\nsubtracting one pointer from another:\n");
35     printf("ptr2 = %p, ptr1 = %p, ptr2 - ptr1 = %td\n",
36           ptr2, ptr1, ptr2 - ptr1);
37     // subtract an integer from a pointer
38     printf("\nsubtracting an int from a pointer:\n");
39     printf("ptr3 = %p, ptr3 - 2 = %p\n",
40           ptr3, ptr3 - 2);
41
42     return 0;
43 }

```

```

1  pointer value, dereferenced pointer, pointer address:
2  ptr1 = 000000d5c5bffb60, *ptr1 =100, &ptr1 = 000000d5c5bffb58
3
4  adding an int to a pointer:
5  ptr1 + 4 = 000000d5c5bffb70, *(ptr4 + 3) = 400
6
7  values after ptr1++:
8  ptr1= 000000d5c5bffb64, *ptr1 =200, &ptr1 = 000000d5c5bffb58
9
10 values after --ptr2:
11 ptr2 = 000000d5c5bffb64, *ptr2 = 200, &ptr2 = 000000d5c5bffb50
12
13 Pointers reset to original values:
14 ptr1 = 000000d5c5bffb60, ptr2 = 000000d5c5bffb68
15
16 subtracting one pointer from another:
17 ptr2 = 000000d5c5bffb68, ptr1 = 000000d5c5bffb60, ptr2 - ptr1 = 2
18
19 subtracting an int from a pointer:

```

pnt_add.c

- 作用: 演示不同数据类型的指针加法规律, 展示指针运算与数据类型大小的关系。
- 功能:
 1. 定义 `short` 与 `double` 类型的数组;
 2. 分别声明对应类型的指针并指向数组首地址;
 3. 通过指针加法 (`ptr + index`) 操作观察地址变化;
 4. 比较不同类型下每次加 1 实际偏移的字节数 (`sizeof(type)`)。
- 演示效果:

```

1 // pnt_add.c -- pointer addition
2 #include <stdio.h>
3 #define SIZE 4
4 int main(void)
5 {
6     short dates [SIZE];
7     short * pti;
8     short index;
9     double bills[SIZE];
10    double * ptf;
11
12    pti = dates;    // assign address of array to pointer
13    ptf = bills;
14    printf("%23s %15s\n", "short", "double");
15    for (index = 0; index < SIZE; index++)
16        printf("pointers + %d: %10p %10p\n",
17              index, pti + index, ptf + index);
18
19    return 0;
20 }
```

```

1 short          double
2 pointers+ 0:  7973ff6a0  7973ff680
3 pointers + 1:  7973ff6a2  7973ff688
4 pointers + 2:  7973ff6a4  7973ff690
5 pointers + 3:  7973ff6a6  7973ff698
```

order.c

- 作用: 演示指针运算中的运算符优先级与结合性, 说明 `p++`、`++p`、`(*p)++` 的区别。
- 功能:
 1. 定义两个整型数组 `data` 与 `moredata` ;
 2. 使用三个指针 `p1`、`p2`、`p3` 分别指向数组元素;
 3. 展示 `*p1++` (先取值再指针递增)、`*++p2` (先指针递增再取值)、`(*p3)++` (先取值再自增该值) 的效果;
 4. 输出每一步操作后的指针指向与值变化。
- 演示效果:

```
1  /* order.c -- precedence in pointer operations */
2  #include <stdio.h>
3  int data[2] = {100, 200};
4  int moredata[2] = {300, 400};
5  int main(void)
6  {
7      int * p1, * p2, * p3;
8
9      p1 = p2 = data;
10     p3 = moredata;
11     printf("  *p1 = %d,   *p2 = %d,     *p3 = %d\n",
12            *p1      ,   *p2      ,     *p3);
13     printf("*p1++ = %d, *++p2 = %d, (*p3)++ = %d\n",
14            *p1++    ,  *++p2     ,  (*p3)++);
15     printf("  *p1 = %d,   *p2 = %d,     *p3 = %d\n",
16            *p1      ,   *p2      ,     *p3);
17
18     return 0;
19 }
```

```
1  *p1 = 100,   *p2 = 100,     *p3 = 300
2  *p1++ = 100, *++p2 = 200, (*p3)++ = 300
3  *p1 = 200,   *p2 = 200,     *p3 = 301
```

no_data.c

- 作用: 演示未初始化的局部数组的默认值情况, 说明其内容为不确定的“垃圾值”。

- 功能:
 1. 定义整型数组 `no_data[SIZE]` , 未进行初始化;
 2. 使用 `for` 循环输出数组每个元素的值;
 3. 展示局部变量在栈上分配时的初始状态;
 4. 强调未初始化变量的不可预测性。
- 演示效果:

```
1  /* no_data.c -- uninitialized array */
2  #include <stdio.h>
3  #define SIZE 4
4  int main(void)
5  {
6      int no_data[SIZE]; /* uninitialized array */
7      int i;
8
9      printf("%2s%14s\n",
10             "i", "no_data[i]");
11     for (i = 0; i < SIZE; i++)
12         printf("%2d%14d\n", i, no_data[i]);
13
14     return 0;
15 }
```

1	i	no_data[i]
2	0	1356386368
3	1	32758
4	2	1356386376
5	3	32758

flc.c

- 作用: 演示复合字面量 (compound literals) 的使用方法, 展示如何在表达式中创建匿名数组。
- 功能:
 1. 使用 `(int [2]){10, 20}` 生成匿名一维数组;
 2. 使用 `(int [2][COLS]){ {1,2,3,-9}, {4,5,6,-8} }` 生成匿名二维数组;
 3. 调用函数 `sum()` 与 `sum2d()` 分别对一维、二维数组计算总和;
 4. 展示静态定义与复合字面量的灵活性。

- 演示效果:

```
1 // flc.c -- funny-looking constants
2 #include <stdio.h>
3 #define COLS 4
4 int sum2d(const int ar[][COLS], int rows);
5 int sum(const int ar[], int n);
6 int main(void)
7 {
8     int total1, total2, total3;
9     int * pt1;
10    int (*pt2)[COLS];
11
12    pt1 = (int [2]) {10, 20};
13    pt2 = (int [2][COLS]) { {1,2,3,-9}, {4,5,6,-8} };
14
15    total1 = sum(pt1, 2);
16    total2 = sum2d(pt2, 2);
17    total3 = sum((int []){4,4,4,5,5,5}, 6);
18    printf("total1 = %d\n", total1);
19    printf("total2 = %d\n", total2);
20    printf("total3 = %d\n", total3);
21
22    return 0;
23 }
24
25 int sum(const int ar[], int n)
26 {
27     int i;
28     int total = 0;
29
30     for( i = 0; i < n; i++)
31         total += ar[i];
32
33     return total;
34 }
35
36 int sum2d(const int ar[][COLS], int rows)
37 {
38     int r;
39     int c;
40     int tot = 0;
41
42     for (r = 0; r < rows; r++)
```

```

43         for (c = 0; c < COLS; c++)
44             tot += ar[r][c];
45
46     return tot;
47 }

```

```

1     total1 = 30
2     total2 = 4
3     total3 = 27

```

designate.c

- 作用: 演示使用 C99 的“指定初始化器” (designated initializers) 为数组特定位置赋值。
- 功能:
 1. 定义整型数组 `days[MONTHS]` 并在初始化列表中使用 `[index] = value` 语法;
 2. 同时混合普通初始化与指定初始化器;
 3. 输出数组中每个月的天数, 观察未显式赋值的元素是否为 0;
 4. 展示指定初始化器的灵活性与覆盖行为。
- 演示效果:

```

1 // designate.c -- use designated initializers
2 #include <stdio.h>
3 #define MONTHS 12
4 int main(void)
5 {
6     int days[MONTHS] = {31, 28, [4] = 31, 30, 31, [1] = 29};
7     int i;
8
9     for (i = 0; i < MONTHS; i++)
10         printf("%2d  %d\n", i + 1, days[i]);
11
12     return 0;
13 }

```

1	1	31
2	2	29
3	3	0
4	4	0
5	5	31
6	6	30
7	7	31
8	8	0
9	9	0
10	10	0
11	11	0
12	12	0

day_mon3.c

- 作用: 演示使用指针表达式访问数组元素, 说明 `*(days + index)` 与 `days[index]` 的等价性。
- 功能:
 1. 定义整型数组 `days[MONTHS]` 存储每个月的天数;
 2. 使用指针偏移法 `*(days + index)` 访问数组元素;
 3. 输出每个月的天数以验证指针与数组的对应关系;
 4. 强调数组与指针在 C 语言中的本质兼容性。
- 演示效果:

```
1  /* day_mon3.c -- uses pointer notation */
2  #include <stdio.h>
3  #define MONTHS 12
4
5  int main(void)
6  {
7      int days[MONTHS] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
8      int index;
9
10     for (index = 0; index < MONTHS; index++)
11         printf("Month %2d has %d days.\n", index + 1,
12             *(days + index));    // same as days[index]
13
14     return 0;
```



```
15 | }
```

```
1 | Month 1 has 31 days.
2 | Month 2 has 28 days.
3 | Month 3 has 31 days.
4 | Month 4 has 30 days.
5 | Month 5 has 31 days.
6 | Month 6 has 30 days.
7 | Month 7 has 31 days.
8 | Month 8 has 31 days.
9 | Month 9 has 30 days.
10 | Month 10 has 31 days.
11 | Month 11 has 30 days.
12 | Month 12 has 31 days.
```

day_mon2.c

- 作用: 演示利用 `sizeof` 运算符自动计算数组元素个数，避免手动定义常量。
- 功能:
 1. 定义常量整型数组 `days[]` 存储每个月的天数；
 2. 使用 `sizeof days / sizeof days[0]` 计算数组长度；
 3. 遍历数组并输出每个月的天数；
 4. 展示 `sizeof` 在数组长度计算中的应用。
- 演示效果:

```
1 | /* day_mon2.c -- letting the compiler count elements */
2 | #include <stdio.h>
3 | int main(void)
4 | {
5 |     const int days[] = {31,28,31,30,31,30,31,31,30,31};
6 |     int index;
7 |
8 |     for (index = 0; index < sizeof days / sizeof days[0]; index++)
9 |         printf("Month %2d has %d days.\n", index +1,
10 |             days[index]);
11 |
12 |     return 0;
13 | }
```

```
1  Month 1 has 31 days.
2  Month 2 has 28 days.
3  Month 3 has 31 days.
4  Month 4 has 30 days.
5  Month 5 has 31 days.
6  Month 6 has 30 days.
7  Month 7 has 31 days.
8  Month 8 has 31 days.
9  Month 9 has 30 days.
10 Month 10 has 31 days.
```

day_mon1.c

- 作用: 演示使用数组存储每个月的天数并依次输出。
- 功能:
 1. 定义整型数组 `days[MONTHS]` 存储 12 个月的天数;
 2. 使用 `for` 循环遍历数组并输出每个月的天数;
 3. 展示数组初始化与遍历的基本用法。
- 演示效果:

```
1  /* day_mon1.c -- prints the days for each month */
2  #include <stdio.h>
3  #define MONTHS 12
4
5  int main(void)
6  {
7      int days[MONTHS] = {31,28,31,30,31,30,31,31,30,31,30,31};
8      int index;
9
10     for (index = 0; index < MONTHS; index++)
11         printf("Month %d has %2d days.\n", index +1,
12               days[index]);
13
14     return 0;
15 }
```

```
1   Month 1 has 31 days.
2   Month 2 has 28 days.
3   Month 3 has 31 days.
4   Month 4 has 30 days.
5   Month 5 has 31 days.
6   Month 6 has 30 days.
7   Month 7 has 31 days.
8   Month 8 has 31 days.
9   Month 9 has 30 days.
10  Month 10 has 31 days.
11  Month 11 has 30 days.
12  Month 12 has 31 days.
```

bounds.c

- 作用: 演示数组越界访问的危险性, 说明 C 语言对下标范围不进行检查。
- 功能:
 1. 定义数组 `arr[SIZE]` 并有意访问边界外的元素;
 2. 通过循环索引 -1 到 SIZE, 展示数组越界赋值行为;
 3. 输出越界访问导致的意外内存改动;
 4. 显示数组元素与其他变量的地址, 说明内存相邻布局。
- 主要错误:
 - 循环访问 `arr[-1]` 与 `arr[SIZE]` 属于越界操作, 会造成意外内存覆盖。
- 演示效果:

```
1   // bounds.c -- exceed the bounds of an array
2   #include <stdio.h>
3   #define SIZE 4
4   int main(void)
5   {
6       int value1 = 44;
7       int arr[SIZE];
8       int value2 = 88;
9       int i;
10
11       printf("value1 = %d, value2 = %d\n", value1, value2);
12       for (i = -1; i ≤ SIZE; i++)
13           arr[i] = 2 * i + 1;
```

```

14
15     for (i = -1; i < 7; i++)
16         printf("%2d %d\n", i , arr[i]);
17     printf("value1 = %d, value2 = %d\n", value1, value2);
18
19     printf("address of arr[-1]: %p\n", &arr[-1]);
20     printf("address of arr[4]: %p\n", &arr[4]);
21     printf("address of value1: %p\n", &value1);
22     printf("address of value2: %p\n", &value2);
23
24     return 0;
25 }

```

```

1  value1 = 44, value2 = 88
2  -1 -1
3   0  1
4   1  3
5   2  5
6   3  7
7   4  9
8   5 567
9   6 44
10 value1 = 44, value2 = -1
11 address of arr[-1]: 0000003e537ff74c
12 address of arr[4]:  0000003e537ff760
13 address of value1:  0000003e537ff768
14 address of value2:  0000003e537ff74c

```

array2d.c

- 作用: 演示二维数组作为函数参数时的不同传递与访问方式。
- 功能:
 1. 定义 3×4 的二维数组 `junk` ;
 2. 调用函数 `sum_rows()` 、 `sum_cols()` 计算行和列的总和;
 3. 调用函数 `sum2d()` 计算所有元素的总和;
 4. 展示二维数组参数的三种函数声明语法差异。
- 演示效果:

```

1 // array2d.c -- functions for 2d arrays

```

```

2  #include <stdio.h>
3  #define ROWS 3
4  #define COLS 4
5  void sum_rows(int ar[][COLS], int rows);
6  void sum_cols(int ar[][COLS], int );    // ok to omit names
7  int sum2d(int (*ar)[COLS], int rows); // another syntax
8  int main(void)
9  {
10     int junk[ROWS][COLS] = {
11         {2,4,6,8},
12         {3,5,7,9},
13         {12,10,8,6}
14     };
15
16     sum_rows(junk, ROWS);
17     sum_cols(junk, ROWS);
18     printf("Sum of all elements = %d\n", sum2d(junk, ROWS));
19
20     return 0;
21 }
22
23 void sum_rows(int ar[][COLS], int rows)
24 {
25     int r;
26     int c;
27     int tot;
28
29     for (r = 0; r < rows; r++)
30     {
31         tot = 0;
32         for (c = 0; c < COLS; c++)
33             tot += ar[r][c];
34         printf("row %d: sum = %d\n", r, tot);
35     }
36 }
37
38 void sum_cols(int ar[][COLS], int rows)
39 {
40     int r;
41     int c;
42     int tot;
43
44     for (c = 0; c < COLS; c++)
45     {

```

```

46         tot = 0;
47         for (r = 0; r < rows; r++)
48             tot += ar[r][c];
49         printf("col %d: sum = %d\n", c, tot);
50     }
51 }
52
53 int sum2d(int ar[][COLS], int rows)
54 {
55     int r;
56     int c;
57     int tot = 0;
58
59     for (r = 0; r < rows; r++)
60         for (c = 0; c < COLS; c++)
61             tot += ar[r][c];
62
63     return tot;
64 }

```

```

1   row 0: sum = 20
2   row 1: sum = 24
3   row 2: sum = 36
4   col 0: sum = 17
5   col 1: sum = 19
6   col 2: sum = 21
7   col 3: sum = 23
8   Sum of all elements = 80

```

arf.c

- 作用: 演示通过函数操作数组，体现数组作为参数传递时的特性。
- 功能:
 1. 定义数组 `dip[SIZE]` 并输出初始值;
 2. 调用函数 `mult_array()` 对每个元素乘以一个倍率;
 3. 使用 `show_array()` 输出调用前后的数组内容;
 4. 演示函数间数组传递与指针运算的效果。
- 演示效果:

```

1  /* arf.c -- array functions */
2  #include <stdio.h>
3  #define SIZE 5
4  void show_array(const double ar[], int n);
5  void mult_array(double ar[], int n, double mult);
6  int main(void)
7  {
8      double dip[SIZE] = {20.0, 17.66, 8.2, 15.3, 22.22};
9
10     printf("The original dip array:\n");
11     show_array(dip, SIZE);
12     mult_array(dip, SIZE, 2.5);
13     printf("The dip array after calling mult_array():\n");
14     show_array(dip, SIZE);
15
16     return 0;
17 }
18
19 /* displays array contents */
20 void show_array(const double ar[], int n)
21 {
22     int i;
23
24     for (i = 0; i < n; i++)
25         printf("%8.3f ", ar[i]);
26     putchar('\n');
27 }
28
29 /* multiplies each array member by the same multiplier */
30 void mult_array(double ar[], int n, double mult)
31 {
32     int i;
33
34     for (i = 0; i < n; i++)
35         ar[i] *= mult;
36 }

```

```

1  The original dip array:
2      20.000   17.660    8.200   15.300   22.220
3  The dip array after calling mult_array():
4      50.000   44.150   20.500   38.250   55.550

```

zippo2.c

- 作用: 演示通过指针变量访问二维数组的方式, 与直接使用数组名的效果对比。
- 功能:
 1. 定义指向含两个元素数组的指针 `int (*pz)[2]` ;
 2. 让 `pz` 指向二维数组 `zippo` ;
 3. 展示 `pz`、`pz[0]`、`*pz`、`pz+1` 等不同层级解引用的关系;
 4. 比较间接访问 `(*(*(pz+2)+1))` 与 `pz[2][1]` 的等价性。
- 演示效果:

```
1  /* zippo2.c --  zippo info via a pointer variable */
2  #include <stdio.h>
3  int main(void)
4  {
5      int zippo[4][2] = { {2,4}, {6,8}, {1,3}, {5, 7} };
6      int (*pz)[2];
7      pz = zippo;
8
9      printf("    pz = %p,    pz + 1 = %p\n",
10             pz,          pz + 1);
11      printf("pz[0] = %p, pz[0] + 1 = %p\n",
12             pz[0],       pz[0] + 1);
13      printf(" *pz = %p,  *pz + 1 = %p\n",
14             *pz,         *pz + 1);
15      printf("pz[0][0] = %d\n", pz[0][0]);
16      printf(" *pz[0] = %d\n", *pz[0]);
17      printf(" **pz = %d\n", **pz);
18      printf("    pz[2][1] = %d\n", pz[2][1]);
19      printf("*(*(pz+2) + 1) = %d\n", *(*(pz+2) + 1));
20
21      return 0;
22 }
```



```

1  pz = 00000004257ffd20,    pz + 1 = 00000004257ffd28
2  pz[0] = 00000004257ffd20, pz[0] + 1 = 00000004257ffd24
3  *pz = 00000004257ffd20,   *pz + 1 = 00000004257ffd24
4  pz[0][0] = 2
5      *pz[0] = 2
6          **pz = 2
7              pz[2][1] = 3
8  *((pz+2) + 1) = 3

```

zippo1.c

- 作用: 演示二维数组在内存中的地址特性、指针运算规律及不同层级的解引用效果。
- 功能:
 1. 展示二维数组 `zippo[4][2]` 的存储结构;
 2. 比较 `zippo`、`zippo[0]`、`*zippo` 等指针表达式的结果;
 3. 通过递增指针及解引用验证地址偏移规律;
 4. 输出多层次指针访问结果及对应数值。
- 演示效果:

```

1  /* zippo1.c -- zippo info */
2  #include <stdio.h>
3  int main(void)
4  {
5      int zippo[4][2] = { {2,4}, {6,8}, {1,3}, {5, 7} };
6
7      printf("    zippo = %p,    zippo + 1 = %p\n",
8             zippo,          zippo + 1);
9      printf("zippo[0] = %p, zippo[0] + 1 = %p\n",
10             zippo[0],       zippo[0] + 1);
11     printf(" *zippo = %p,   *zippo + 1 = %p\n",
12            *zippo,          *zippo + 1);
13     printf("zippo[0][0] = %d\n", zippo[0][0]);
14     printf(" *zippo[0] = %d\n", *zippo[0]);
15     printf(" **zippo = %d\n", **zippo);
16     printf("        zippo[2][1] = %d\n", zippo[2][1]);
17     printf("*((zippo+2) + 1) = %d\n", *((zippo+2) + 1));
18
19     return 0;
20 }

```

```

1  zippo = 0000004d9a5ffdd0,    zippo + 1 = 0000004d9a5ffdd8
2  zippo[0] = 0000004d9a5ffdd0, zippo[0] + 1 = 0000004d9a5ffdd4
3      *zippo = 0000004d9a5ffdd0,    *zippo + 1 = 0000004d9a5ffdd4
4  zippo[0][0] = 2
5      *zippo[0] = 2
6      **zippo = 2
7          zippo[2][1] = 3
8  *((*(zippo+2) + 1) = 3

```

vararr2d.c

- 作用: 演示变长数组（VLA, Variable Length Array）的定义与使用，展示其在函数形参中的应用。
- 功能:
 1. 使用不同大小的二维数组（固定与变长）；
 2. 利用函数 `sum2d()` 接收变长二维数组参数；
 3. 演示编译器如何通过传递行列数实现灵活维度计算；
 4. 输出不同尺寸矩阵的元素和。
- 演示效果:

```

1  //vararr2d.c -- functions using VLAs
2  #include <stdio.h>
3  #define ROWS 3
4  #define COLS 4
5  int sum2d(int rows, int cols, int ar[rows][cols]);
6  int main(void)
7  {
8      int i, j;
9      int rs = 3;
10     int cs = 10;
11     int junk[ROWS][COLS] = {
12         {2,4,6,8},
13         {3,5,7,9},
14         {12,10,8,6}
15     };
16
17     int morejunk[ROWS-1][COLS+2] = {
18         {20,30,40,50,60,70},
19         {5,6,7,8,9,10}
20     };

```

```

21
22     int varr[rs][cs]; // VLA
23
24     for (i = 0; i < rs; i++)
25         for (j = 0; j < cs; j++)
26             varr[i][j] = i * j + j;
27
28     printf("3x5 array\n");
29     printf("Sum of all elements = %d\n",
30           sum2d(ROWS, COLS, junk));
31
32     printf("2x6 array\n");
33     printf("Sum of all elements = %d\n",
34           sum2d(ROWS-1, COLS+2, morejunk));
35
36     printf("3x10 VLA\n");
37     printf("Sum of all elements = %d\n",
38           sum2d(rs, cs, varr));
39
40     return 0;
41 }
42
43 // function with a VLA parameter
44 int sum2d(int rows, int cols, int ar[rows][cols])
45 {
46     int r;
47     int c;
48     int tot = 0;
49
50     for (r = 0; r < rows; r++)
51         for (c = 0; c < cols; c++)
52             tot += ar[r][c];
53
54     return tot;
55 }

```

```

1  3x5 array
2  Sum of all elements = 80
3  2x6 array
4  Sum of all elements = 315
5  3x10 VLA
6  Sum of all elements = 270

```

sum_arr2.c

- 作用: 演示使用指针运算遍历数组并求和的过程, 体现 C 语言中指针与数组的紧密关系。
- 功能:
 1. 定义整型数组 `marbles[]` 并初始化;
 2. 在 `sump()` 函数中使用两个指针参数表示数组的起始和结束位置;
 3. 通过 `while (start < end)` 循环遍历数组元素并累加;
 4. 展示指针递增与解引用操作。
- 演示效果:

```
1  /* sum_arr2.c -- sums the elements of an array */
2  #include <stdio.h>
3  #define SIZE 10
4  int sump(int * start, int * end);
5  int main(void)
6  {
7      int marbles[SIZE] = {20,10,5,39,4,16,19,26,31,20};
8      long answer;
9
10     answer = sump(marbles, marbles + SIZE);
11     printf("The total number of marbles is %ld.\n", answer);
12
13     return 0;
14 }
15
16 /* use pointer arithmetic */
17 int sump(int * start, int * end)
18 {
19     int total = 0;
20
21     while (start < end)
22     {
23         total += *start; // add value to total
24         start++;         // advance pointer to next element
25     }
26
27     return total;
28 }
```

1 | The total number of marbles is 190.

sum_arr1.c

- 作用: 演示在函数中传递数组时, 形参退化为指针的特性, 比较 `sizeof` 运算符在不同作用域中的表现。
- 功能:
 1. 主函数中定义整型数组 `marbles[]`, 含 10 个元素;
 2. 在 `sum()` 函数中统计数组元素总和;
 3. 比较在主函数与子函数中使用 `sizeof` 的返回值;
 4. 展示数组传参时 C 语言的“指针退化”机制。
- 语义错误:
 - 使用 `sizeof ar` 在函数中获取数组大小是不正确的, 因为在函数形参处 `ar` 被视为指针, 返回的是指针大小而非数组长度。
- 演示效果:

```
1 // sum_arr1.c -- sums the elements of an array
2 // use %u or %lu if %zd doesn't work
3 #include <stdio.h>
4 #define SIZE 10
5 int sum(int ar[], int n);
6 int main(void)
7 {
8     int marbles[SIZE] = {20,10,5,39,4,16,19,26,31,20};
9     long answer;
10
11     answer = sum(marbles, SIZE);
12     printf("The total number of marbles is %ld.\n", answer);
13     printf("The size of marbles is %zd bytes.\n",
14           sizeof marbles);
15
16     return 0;
17 }
18
19 int sum(int ar[], int n)    // how big an array?
20 {
21     int i;
22     int total = 0;
```

```

23
24     for( i = 0; i < n; i++)
25         total += ar[i];
26     printf("The size of ar is %zd bytes.\n", sizeof ar);
27
28     return total;
29 }

```

```

1  sum_arr1.c: In function 'sum':
2  sum_arr1.c:26:53: warning: 'sizeof' on array function parameter 'ar' will
   return size of 'int *' [-Wsizeof-array-argument]
3      26 |     printf("The size of ar is %zd bytes.\n", sizeof ar);
4          |     ^~
5  sum_arr1.c:19:13: note: declared here
6      19 | int sum(int ar[], int n)    // how big an array?
7          | ~~~~^~~~
8  The size of ar is 8 bytes.
9  The total number of marbles is 190.
10 The size of marbles is 40 bytes.

```

some_data.c

- 作用: 演示部分初始化数组的行为, 说明未显式初始化的元素会被自动赋值为 0。
- 功能:
 1. 定义数组 `some_data[4]`, 仅给前两项赋值;
 2. 输出每个元素的值, 观察剩余未赋值项是否为 0;
 3. 展示 C 语言数组初始化规则。
- 演示效果:

```

1  /* some_data.c -- partially initialized array */
2  #include <stdio.h>
3  #define SIZE 4
4  int main(void)
5  {
6      int some_data[SIZE] = {1492, 1066};
7      int i;
8
9      printf("%2s%14s\n",
10             "i", "some_data[i]");

```

```

11     for (i = 0; i < SIZE; i++)
12         printf("%2d%14d\n", i, some_data[i]);
13
14     return 0;
15 }

```

```

1  i  some_data[i]
2  0      1492
3  1      1066
4  2         0
5  3         0

```

rain.c

- 作用: 计算多年度降雨量总和、平均值及每月平均降雨量，展示二维数组的实际应用。
- 功能:
 1. 定义 5 年 (2010–2014) 的 12 个月降雨量数组;
 2. 逐年计算年度总降雨量并求平均;
 3. 计算每月跨年平均降雨量;
 4. 将结果按格式输出。
- 演示效果:

```

1  /* rain.c -- finds yearly totals, yearly average, and monthly
2     average for several years of rainfall data */
3  #include <stdio.h>
4  #define MONTHS 12    // number of months in a year
5  #define YEARS 5      // number of years of data
6  int main(void)
7  {
8      // initializing rainfall data for 2010 - 2014
9      const float rain[YEARS][MONTHS] =
10     {
11         {4.3,4.3,4.3,3.0,2.0,1.2,0.2,0.2,0.4,2.4,3.5,6.6},
12         {8.5,8.2,1.2,1.6,2.4,0.0,5.2,0.9,0.3,0.9,1.4,7.3},
13         {9.1,8.5,6.7,4.3,2.1,0.8,0.2,0.2,1.1,2.3,6.1,8.4},
14         {7.2,9.9,8.4,3.3,1.2,0.8,0.4,0.0,0.6,1.7,4.3,6.2},
15         {7.6,5.6,3.8,2.8,3.8,0.2,0.0,0.0,0.0,1.3,2.6,5.2}
16     };
17     int year, month;

```

```

18     float subtot, total;
19
20     printf(" YEAR      RAINFALL  (inches)\n");
21     for (year = 0, total = 0; year < YEARS; year++)
22     {
23         // for each year, sum rainfall for each month
24         for (month = 0, subtot = 0; month < MONTHS; month++)
25             subtot += rain[year][month];
26         printf("%5d %15.1f\n", 2010 + year, subtot);
27         total += subtot; // total for all years
28     }
29     printf("\nThe yearly average is %.1f inches.\n\n",
30           total/YEARS);
31     printf("MONTHLY AVERAGES:\n\n");
32     printf(" Jan  Feb  Mar  Apr  May  Jun  Jul  Aug  Sep  Oct ");
33     printf(" Nov  Dec\n");
34
35     for (month = 0; month < MONTHS; month++)
36     {
37         // for each month, sum rainfall over years
38         for (year = 0, subtot = 0; year < YEARS; year++)
39             subtot += rain[year][month];
40         printf("%4.1f ", subtot/YEARS);
41     }
42     printf("\n");
43     return 0;
44 }

```

```

1  YEAR      RAINFALL  (inches)
2  2010              32.4
3  2011              37.9
4  2012              49.8
5  2013              44.0
6  2014              32.9
7
8  The yearly average is 39.4 inches.
9
10 MONTHLY AVERAGES:
11
12  Jan  Feb  Mar  Apr  May  Jun  Jul  Aug  Sep  Oct  Nov  Dec
13  7.3  7.3  4.9  3.0  2.3  0.6  1.2  0.3  0.5  1.7  3.6  6.7
14

```